

Azure Data Factory - Interview Prep

Azure Data Factory - Interview Questions

Based on your adf007 repo structure: dataset | factory | integrationRuntime | linkedService | pipeline | trigger | templates/Parent Pipeline Orchestration | publish_config.json

1. Core ADF Building Blocks (tied to your repo folders)

Q1: Your repo has separate linkedService, dataset, and pipeline folders. Explain the relationship between these three. - **Linked Service** = the connection string/credentials to a source or target (e.g., Azure SQL, Blob Storage, ADLS Gen2, Snowflake). - **Dataset** = a named reference to a specific structure/location within that linked service (e.g., a table or a folder path). - **Pipeline** = the orchestration logic – activities (Copy, Lookup, ForEach, etc.) that consume datasets to move/transform data. One linked service can be reused by many datasets; one dataset can be reused by many pipelines.

Q2: What is an Integration Runtime (IR), and which type would you use for on-prem vs. cloud sources? - **Azure IR**: fully managed, for cloud-to-cloud movement (e.g., Blob -> Snowflake). - **Self-Hosted IR**: installed on-prem/VM, needed when source/sink sits behind a firewall or in a private network (e.g., on-prem SQL Server, or Snowflake via private endpoint). - **Azure-SSIS IR**: only if lifting-and-shifting legacy SSIS packages. *Be ready to say which one your project actually used and why (private network vs. public endpoint access).*

Q3: What's in publish_config.json, and why does it matter for deployment? It holds environment-specific ARM template parameters (connection strings, IR names, etc.) used when **publishing/deploying** the ADF instance across environments (Dev/QA/Prod) via CI/CD – this is what lets the same pipeline JSON deploy differently per environment without hardcoding values.

Q4: Your repo has a templates/Parent Pipeline Orchestration folder. Explain this design pattern. This is a **metadata-driven framework**: a single “parent” pipeline reads a **control/metadata table** (e.g., listing source tables, target tables, load type, active flag) via a **Lookup** activity, loops through it with a **ForEach**, and inside the loop calls an **Execute Pipeline** activity to trigger a generic “child” pipeline for each entry – instead of hand-building one pipeline per table. *Why it matters*: massively reduces pipeline sprawl when ingesting dozens/hundreds of tables (very similar in spirit to your Snowflake project's Python framework that auto-generates pipelines from a schema file – a great parallel to draw in the interview).

Q5: What does a trigger do, and what types exist? - **Schedule trigger** – runs on a fixed time/cron-like schedule. - **Tumbling window trigger** – time-sliced, supports backfill/retry per window, tracks dependency between windows – best for sequential incremental loads. - **Event-based trigger** – fires on a Blob create/delete event (e.g., new file lands in ADLS/S3-equivalent) – closest analog to your Snowflake project's file-drop-triggered pattern. *Be ready to say which trigger type your ADF pipelines used and why.*

2. Data Movement, Transformation & Patterns

Q6: Difference between Copy Activity and Mapping Data Flow? -

Copy Activity: fast, code-free data movement (source -> sink), minimal transformation (column mapping, type conversion). -

Mapping Data Flow: visual, Spark-based transformation engine inside ADF - joins, aggregations, derived columns, surrogate keys - used when transformation logic is too complex for Copy Activity alone.

Q7: How would you implement incremental/delta loads in ADF?

Typical pattern: a **watermark table** storing the last successfully loaded timestamp/ID -> **Lookup** activity reads the watermark ->

Copy Activity filters source query using that watermark -> on success, **Stored Procedure/Set Variable** activity updates the watermark. Compare/contrast with your Snowflake project's MERGE-based delta approach - same concept, different engine.

Q8: How do you parameterize pipelines/datasets for reuse

across multiple tables? Define pipeline parameters (e.g., tableName, schemaName) -> pass them into dataset parameters -> dataset's file path/table name becomes dynamic (@dataset().tableName). This is exactly what enables the Parent Pipeline Orchestration pattern in Q4.

Q9: How do you handle errors/retries in a pipeline?

Activity-level retry and retryIntervalInSeconds settings; **failure path** connections (red arrow) to route to a notification/logging activity (e.g., Web Activity to Teams/Email, or a Stored Procedure to log failure into an audit table); pipeline-level **"On Failure"** dependency conditions.

Q10: If ADF is landing data into Blob/ADLS, and Snowflake is the target - how does that final load typically happen?

Two common patterns: (a) ADF Copy Activity writes directly to Snowflake using a **Snowflake linked service/connector**, or (b) ADF lands files in Blob/ADLS and a separate mechanism (Snowpipe, or your Python-framework-driven COPY INTO) picks them up - ties directly to your BHP project pattern of S3 file-drop -> framework -> COPY INTO. Be ready to state which pattern your actual project used.

Q11: How is CI/CD done for ADF specifically - what's different from a typical app deployment?

ADF's Git-integrated **"collaboration branch"** holds human-readable JSON (factory/pipeline/dataset/linkedService/trigger folders - exactly your repo layout); clicking **"Publish"** generates an adf_publish branch with ARM templates + publish_config.json; CI/CD pipelines (Azure DevOps/GitHub Actions) then deploy those ARM templates across environments using environment-specific parameter overrides.

Q12 (Likely asked to test depth): What's the difference between "Publish" and a CI/CD release pipeline in ADF?

"Publish" (manual, from ADF Studio) only pushes to the **Dev/Live** ADF instance tied to the collaboration branch. For QA/Prod, you don't click Publish in each environment - instead a **release pipeline** deploys the ARM template artifacts (from adf_publish) directly into the target Data Factory instance via automation, which is the enterprise-safe, auditable approach.

Quick Talking-Point Bridge to Your Snowflake Project

If asked "how does your ADF experience relate to your current Snowflake work," a strong answer: *"Conceptually identical - a metadata-driven parent/child pipeline pattern in ADF is the same idea as our Python framework auto-generating pipeline and DAG files from a schema file in the Snowflake project. Both avoid hand-building one pipeline per source table."*